



МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И
ТЕХНОЛОГИЧЕСКОГО ОБРАЗОВАНИЯ**
Кафедра информационных технологий и электронного обучения

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

«Разработка веб-портфолио студента на примере дисциплины “Программирование”»

Обучающейся 4 курса
Гиниятуллиной Юлии Сергеевны

Научный руководитель:
кандидат физико-математических наук,
доцент кафедры ИТиЭО
Жуков Николай Николаевич

Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
Глава 1. Анализ предметной области и используемых инструментов.....	5
1.1 Анализ предметной области.....	5
1.2 Выбор инструмента для разработки электронного портфолио.....	8
Глава 2. Разработка веб-портфолио студента и описание процесса его создания...	
11	
2.1 Установка и базовая настройка генератора статических сайтов Hugo.....	11
2.2 Кастомизация базового сайта под портфолио студента, наполнение дисциплины «Программирование».....	13
2.3 Размещение сайта на Github Pages.....	19
ЗАКЛЮЧЕНИЕ.....	21
ЛИТЕРАТУРА.....	23
ПРИЛОЖЕНИЕ А.....	26

ВВЕДЕНИЕ

Современная образовательная среда всё активнее интегрирует цифровые технологии, что облегчает жизнь студентам и преподавателям. Все практические и проектные работы можно хранить в электронном формате — как локально, так и в специальных системах управления обучением. Однако при необходимости продемонстрировать результаты обучения широкой аудитории в сети Интернет — возникает потребность в создании единой, интуитивно понятной и структурированной платформы для представления достижений.

Одним из эффективных инструментов представления индивидуальных достижений и работ студента является электронное портфолио, которое отражает уровень его профессиональной подготовки и динамику развития компетенций.

Для студентов технических направлений, создание веб-портфолио особенно актуально, поскольку оно позволяет наглядно продемонстрировать навыки программирования, владения современными технологиями и средствами разработки [1]. Электронное портфолио может использоваться как средство взаимодействия с преподавателями, файловый сервер для хранения выполненных работ, а также может быть хорошим дополнением к резюме при трудоустройстве на работу.

Актуальность темы курсовой работы обусловлена необходимостью систематизации учебных достижений студентов в электронном формате и создания инструмента для их наглядного представления с использованием современных технологий.

Целью курсовой работы является разработка веб-портфолио студента по дисциплине «Программирование» с использованием современных веб-технологий.

Реализация данной цели обусловила необходимость решения следующих **задач**:

- проанализировать понятие электронного портфолио студента, его виды, структуру и требования к содержанию для обучающихся технических направлений;
- провести сравнительный анализ современных инструментов для создания веб-портфолио;
- разработать структуру веб-портфолио с разделением по курсам и дисциплинам;
- осуществить наполнение портфолио по дисциплине «Программирование» с использованием внешних платформ для размещения исходного кода;
- выполнить кастомизацию внешнего вида сайта и настройку конфигурационных параметров, разместить готовое веб-портфолио в сети Интернет.

Объект исследования — электронное портфолио студента как средство представления и систематизации результатов учебной деятельности в цифровой образовательной среде.

Предмет исследования — процесс проектирования, разработки, настройки и размещения веб-портфолио студента по дисциплине «Программирование» с использованием современных веб-технологий.

Практическая значимость исследования заключается в разработке инструкции по созданию портфолио на примере дисциплины «Программирование». Она может быть использована другими обучающимися для представления собственных работ и достижений за все время обучения в университете.

Курсовая работа на тему: «Разработка веб-портфолио студента на примере дисциплины “Программирование”» состоит из введения, двух глав и заключения; содержит n страниц, n рисунков, 1n таблицу, библиографический список, содержащий n источников.

Глава 1. Анализ предметной области и используемых инструментов

1.1 Анализ предметной области

Портфолио (англ. portfolio портфель, папка для важных дел или документов) — собрание документов, образцов работ, фотографий, дающих представление о предлагаемых возможностях, услугах фирмы или специалиста (обычно художника, архитектора, фотографа, фотомодели, журналиста и др.); собрание, совокупность работ, осуществленных проектов и т. п. (фотографа, дизайнера и т. п.) [2, 3].

Исходя из терминологии, можно сделать вывод, что портфолио студента — это совокупность работ, осуществленных проектов и достижений обучающегося, помогающая оценить его результаты за время учебной деятельности.

Еще одна функция портфолио — накопить профессиональный авторский материал для знакомства с будущим работодателем. Собрание всех работ, проектов и достижений студента может быть представлено как в бумажном, так и в электронном виде. У обоих видов есть свои требования к оформлению и содержанию.

Цель портфолио студента — собрать в одном месте всю проделанную работу студента за весь период обучения, чтобы в полной мере отразить навыки, личные качества и достижения обучающегося. Собирать его следует с первого курса, чтобы к последнему курсу оно уже было наполнено. Чем более подробным оно будет сделано, тем проще будет преподавателю искать конкретную работу, а работодателям будет легче найти вас и выделить из толпы.

Существует несколько видов портфолио студента [4]:

1. **Документальное.** Оно содержит официальные документы, подтверждающие результаты деятельности студента: грамоты, сертификаты, награды, дипломы и т.д.
2. **Портфолио отзывов.** Оно содержит положительные отзывы руководителей и преподавателей, а также собственные достижения студента.
3. **Оценочное.** Оно содержит контрольные, тесты, коллоквиумы, самостоятельные и лабораторные работы; создается с целью объективной оценки результатов деятельности студента.
4. **Показательное.** Оно содержит лучшие работы, достижения, которые наглядно демонстрируют навыки, опыт и потенциал студента. Это портфолио будет полезным при трудоустройстве, так как позволяет быстро оценить квалификацию и результаты учебной деятельности.
5. **Портфолио процесса.** Оно демонстрирует степень участия студента в разных проектах и мероприятиях: творческих, учебных, исследовательских, социальных. Документ включает подробное описание всех этапов работы учащегося, выводы по проведенным действиям.
6. **Электронное.** Оно предназначено для фиксации, накопления и оценки результатов деятельности и достижений студента с помощью информационных технологий. Такое портфолио представлено в виде сайта-визитки, информацию на котором можно постоянно пополнять и редактировать. В последнее время самый популярный, гибкий и удобный способ демонстрации знаний и навыков.

Электронное портфолио студента технических и инженерных специальностей должно включать в себя [5]:

- автобиографию студента, в которой можно также указать свои hard / soft skills и технологический стек;
- разделение портфолио по курсам и / или предметам, если необходимо — более детальное разбиение на подразделы;

- документальное портфолио;
- оценочное портфолио;
- работы за весь период обучения;
- научно-исследовательские работы и публикации статей студента, если они есть;
- дополнительное образование, если оно есть;
- портфолио отзывов;
- контактные данные, такие как номер телефона, профиль в Github, Telegram и т.д..

Проанализировав несколько портфолио студентов был сделан вывод, что все работы должны быть представлены последовательно в хронологическом порядке. У каждой работы должно быть краткое описание цели ее выполнения. Необходимо указывать даты получения сертификатов, дипломов, а также участия в мероприятиях. В портфолио следует вести записи регулярно и вносить все виды информации (скринкасты, рисунки, скриншоты, инструкции, элементы кода, ссылки на облачные хранилища / проекты на Github и т.д.) [6, 7].

1.2 Выбор инструмента для разработки электронного портфолио

На современном рынке представлен широкий спектр инструментов и сервисов для создания веб-портфолио, которые можно условно разделить на несколько категорий: конструкторы сайтов, системы управления контентом и генераторы статических сайтов [8, 9].

Конструкторы сайтов (no-code платформы) — это сервис, который позволяет любому пользователю интернета самостоятельно создать веб-сайт, при этом не обладая знаниями в области программирования, верстки и дизайна. Такой способ создания сайта удобен для тех, кто не имеет знаний в сфере создания сайтов или для тех, кому необходимо его сделать в кратчайшие сроки. Популярные конструкторы сайтов: Craftum, Nethouse, Tilda, Wix, uKit. (статья)

При помощи конструктора можно создать сайт используя готовые шаблоны и визуальный редактор на основе drag-and-drop. Процесс работы с конструктором очень прост - пользователь прямо на сайте услуги выбирает структуру (набор страниц), расположение блоков (элементов текста, графики, видео и аудио) и оформление для своего будущего проекта. С помощью конструктора можно создать свои портфолио, лендинг, блог и небольшой интернет-магазин. Однако студента инженерной специальности портфолио, созданное на конструкторе сайтов, вовсе не красит. Его всё еще можно представить и оно имеет право на существование, но, например, большого отклика от работодателей студент вряд ли получит.

Системы управления контентом (расшифровка аббревиатуры CMS — Content Management System) — это программное обеспечение, которое позволяет легко создавать, редактировать, публиковать и управлять содержимым веб-сайта, не требуя глубоких знаний в программировании. Процесс работы с CMS также прост благодаря удобному графическому

интерфейсу и готовым шаблонам. Наиболее популярные CMS: WordPress, Joomla, Drupal, OpenCart, 1С-Битрикс.

Основное отличие CMS от конструктора сайтов заключается в том, что первое является движком сайта, который устанавливается на свой хостинг, а второе арендует платформу и её функционал. Условно говоря, если конструктор вдруг прекратит работу, то сайт пропадет вместе с ним. С CMS такого произойти не может, поскольку все компоненты физически устанавливаются на используемое устройство. Для студента инженерной специальности использование CMS будет презентабельнее, чем конструктора сайтов. Существует ряд недостатков использования систем управления контентом для создания портфолио: необходимость технических навыков для настройки, риски безопасности, «тяжелый» код шаблонов, замедляющий сайт, а также шаблонность, которая мешает выделиться.

Генераторы статических сайтов (расшифровка аббревиатуры SSG — Static Site Generators) — это программные инструменты для создания веб-сайтов, которые превращают исходный контент (обычно в виде текстовых файлов в форматах Markdown, HTML или YAML) в готовый набор статичных HTML-страниц. Наиболее популярные генераторы: Jekyll, Hugo, Pelican, Eleventy [10, 11].

Данные запрашиваются один раз на этапе сборки сайта, после чего пользователю становятся доступны уже готовые страницы сайта, в отличие от CMS, которые генерируют страницы динамически при каждом запросе пользователя. Процесс работы с SSG является наиболее технически ориентированным и удобным. Среди преимуществ такого способа создания сайтов стоит отметить высокую производительность, безопасность и простоту хостинга. Такой способ создания сайта является идеальным для студента инженерной специальности, так как требует знаний в области

программирования (как минимум для хостинга сайта) и выглядит презентабельнее предыдущих способов.

Для создания сайта портфолио был выбран генератор статических сайтов Hugo. Он является оптимальным выбором для студента программиста за счет высокой скорости сборки сайта, простоты установки и хостинга на бесплатных платформах (например: Github Pages). Более подробное обоснование выбора этого программного инструмента представлено в таблице 1.

Таблица 1. Сравнительный анализ генераторов статических сайтов

	Jekyll	Hugo	Pelican	Eleventy
Скорость генерации	Средняя	Высокая	Средняя	Средняя
Простота установки	Средняя	Высокая	Средняя	Средняя
Поддержка Markdown	+	+	+	+
Количество готовых тем	Среднее	Большое	Маленькое	Среднее
Документация	Достаточная	Подробная и актуальная	Ограниченная	Достаточная
Порог входа для студента	Средний	Низкий	Средний	Средний
Расширяемость и кастомизация	Средняя	Высокая	Средняя	Высокая
Стоимость	open-source	open-source	open-source	open-source

Глава 2. Разработка веб-портфолио студента и описание процесса его создания

2.1 Установка и базовая настройка генератора статических сайтов Hugo

На первом этапе практической части осуществляется подготовка рабочей среды и установка генератора статических сайтов Hugo. Для работы с Hugo на локальное устройство также необходимо установить редактор исходного кода Visual Studio Code (VS Code) и иметь личный аккаунт на Github.

Существует несколько способов установки Hugo на компьютер, каждый из которых зависит от имеющейся операционной системы. В рамках курсовой работы использовалась операционная система Windows 10 Pro и был выбран способ установки через менеджер пакетов Chocolatey. Чтобы установить Chocolatey, необходимо перейти на официальный сайт менеджера пакетов по адресу <https://chocolatey.org/install#individual>, выбрать способ Individual, скопировать предлагаемую команду, запустить Powershell от имени администратора и вставить её. Далее необходимо прописать команду “choco install hugo-extended -confirm”, чтобы установить Hugo. Успешную установку можно проверить командой “hugo version”, результат которой будет иметь вид “hugo x0..... windows/amd64”. Далее необходимо создать новый публичный репозиторий на Github, клонировать его на локальный компьютер и открыть с помощью VS Code [12].

В редакторе кода необходимо вызвать терминал, после чего выполнить команду по созданию нового проекта “hugo new site .\ --force”. После выполнения команды создается структура будущего сайта, представленная на рисунке 1.

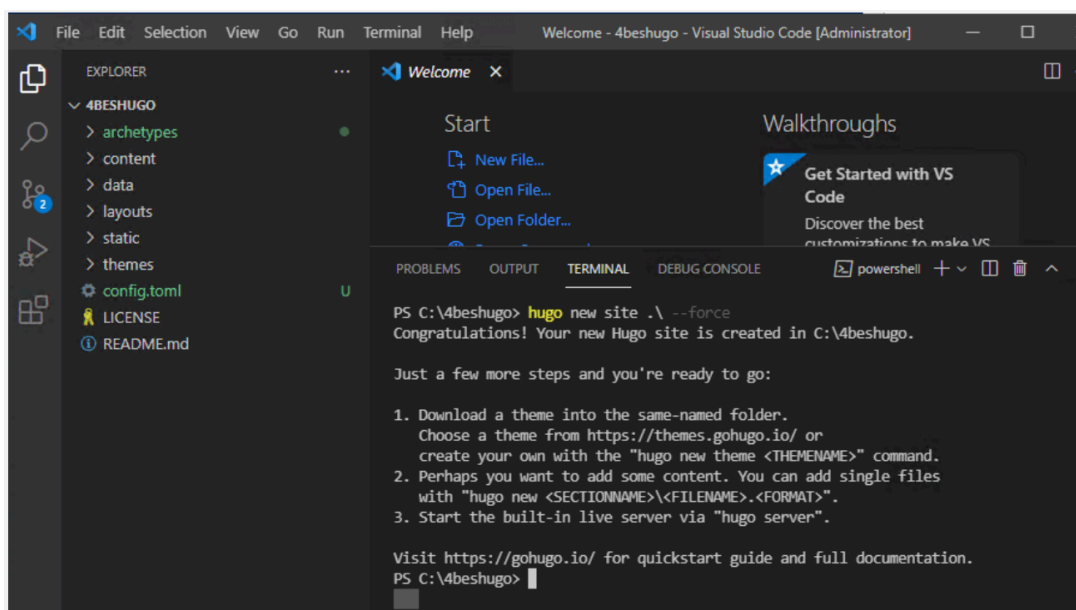


Рисунок 1. Структура будущего сайта

Теперь можно приступить к кастомизации сайта. Основную тему страниц сайта можно как создать самостоятельно, так и воспользоваться готовыми решениями. В рамках курсовой работы была выбрана готовая тема “hello-friend-ng”. Данную тему можно установить командой “git submodule add <https://github.com/rhazdon/hugo-theme-hello-friend-ng.git> themes/hello-friend-ng”. Выбор именно этой темы обусловлен её простотой, минималистичностью и отсутствием перегруженности различными иконками. Портфолио студента должно быть сдержанным, лаконичным и иметь четкую структуру. При оформлении стоит избегать перегруженности информацией и изображениями, разрозненных стилей [13].

В файл “hugo.toml” необходимо добавить строку “theme = ‘hello-friend-ng’”, которая применит к сайту выбранную тему. Далее можно проверить работоспособность сайта и успешную установку темы при помощи команды “Hugo Server”. Результатом этой команды будет “Web Server is available at <http://localhost:1313/> (bind address 127.0.0.1)”. Нажатие по ссылке перенесет на страницу локально размещенного сайта с примененной темой [14].

2.2 Кастомизация базового сайта под портфолио студента, наполнение дисциплины «Программирование»

В Hugo сайт заполняется путем создания содержимого в директории “content”. Для корректного и более читабельного отображения работ по дисциплине «Программирование» было решено структурировать сайт по курсам. Каждый курс будет являться директорией внутри “content”. Внутри каждой директории курса расположены дисциплины, пройденные за 2 семестра, в виде файлов формата “.md”.

Дисциплина «Программирование» началась со 2 семестра 1 курса, где изучался язык C. На последующих курсах изучался язык Python. Отображать на сайте локально расположенные папки и архивы с исходным кодом является плохой практикой, так как они могут быть удалены самостоятельно или в рамках выхода компьютера из строя. Рекомендуется хранить работы по дисциплине «Программирование» на специальных платформах, таких как Github, Replit, Google Collaboratory и т. д.

Работы с использованием языка C было решено расположить на платформе Replit, потому что они легковесные, умецаются в один — два файла, имеют короткое описание сути выполнения и не требуют дополнительных зависимостей. Работы с использованием языка Python выполнялись локально на компьютере, потому было решено создать под каждую работу отдельный репозиторий на Github. Эти работы более тяжеловесные, требуют множество зависимостей. Работы на языке Python про создание и обучение нейросетей выполнялись в Google Collaboratory, так как преподавателю их очень легко проверить, не загружая себе ничего на компьютер. Они требуют много оперативной памяти, чему не всегда может соответствовать устройство преподавателя или студента.

Создание портфолио начинается с создания файла `about_me.md`. В нем следует написать о себе, разместить свою фотографию. Если ваша фотография занимает всю страницу — необходимо прибегнуть к встраиванию HTML в Markdown. Для этого в файле следует вписать строку `" height="<длина>"/>`. Также здесь можно разместить портфолио достижений и свои контакты, ссылки на соцсети.

После этого необходимо создать 4 директории курсов, так как дисциплина «Программирование» началась со 2 семестра 1 курса: `first_course`, `second_course`, `third_course`, `forth_course`. Внутри каждой директории были созданы файлы формата “.md”, где будут храниться работы: `prog_2.md`, `prog_3-4.md`, `prog_5-6.md`, `prog_7.md`. Вид итоговой структуры директории `content` представлен на рисунке 2.

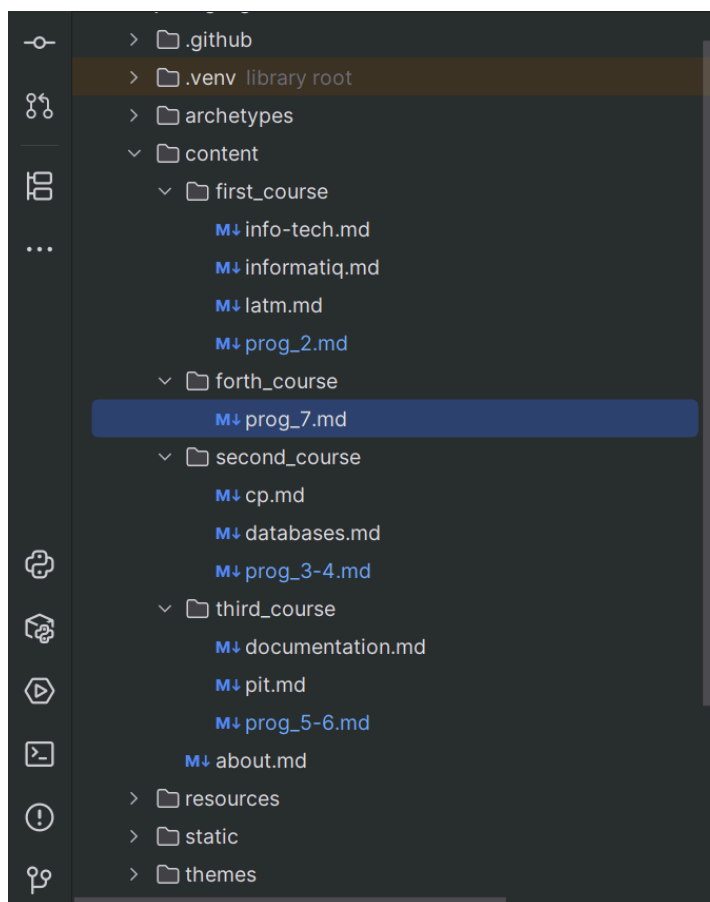


Рисунок 2. Часть итоговой структуры сайта, наполнение директории `content`

Лабораторные работы в файлах будут представлены в виде [<название лабораторной работы>](<ссылка на код проекта>). Таким образом, каждая лабораторная работа представлена в виде названия, которое является гиперссылкой на платформу с размещенным кодом. Название работ отражает их суть [15].

В лабораторных работах на языке C на платформе Replit представлено более детальное описание цели в разделе, так как в рамках одной работы было несколько подпунктов, которые были реализованы как отдельное задание. Пример лабораторной работы в Replit можно увидеть на рисунке 3.

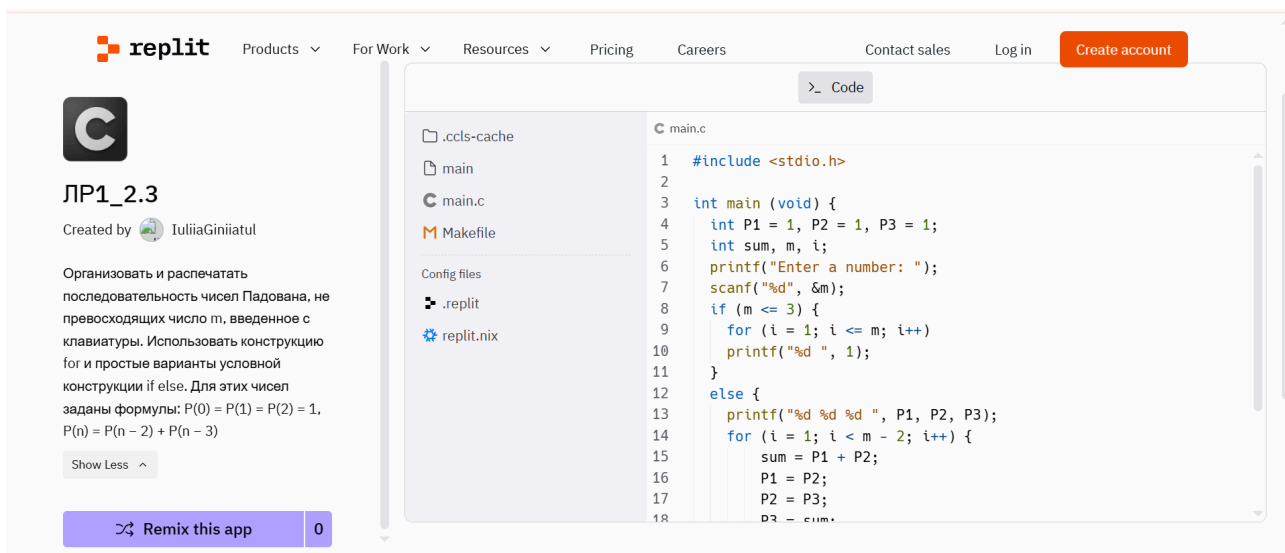


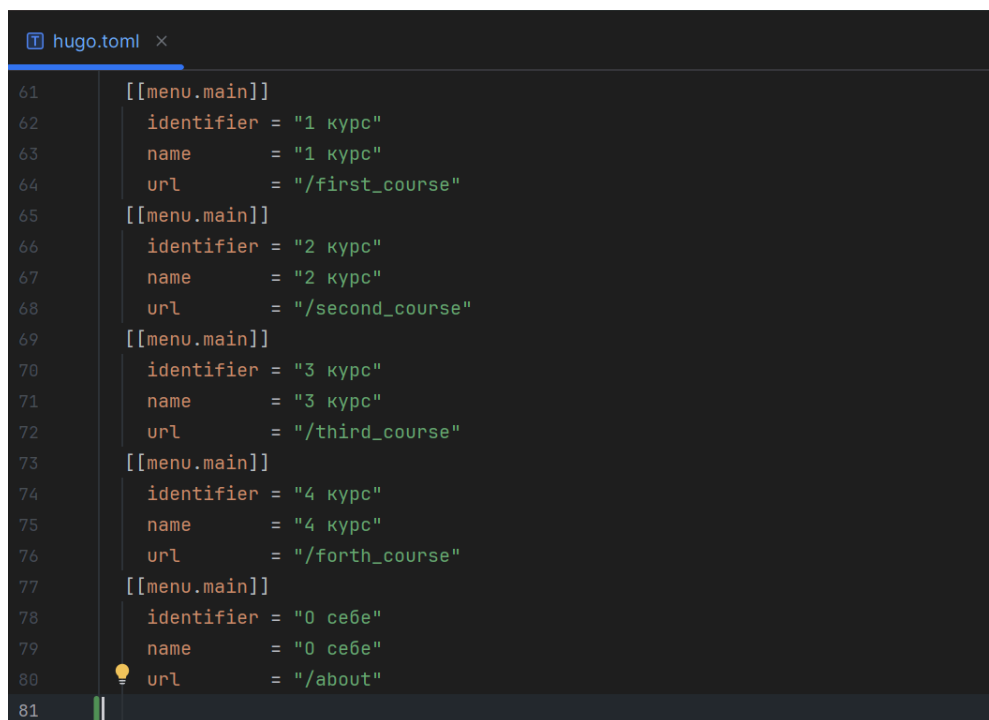
Рисунок 3. Пример оформления лабораторной работа №1, пункт 2.3

Чтобы не путаться в работах, необходимо разделить их по семестрам. Внутри курсов можно было сделать разделение по семестрам, однако, это не всегда удобно. Работы разных семестров было решено разделить внутри заголовками второго уровня “Лабораторные работы ... семестра”. Пример оформления файла с лабораторными работами представлен на рисунке 4.

```
M4 prog_3-4.md x
1 ---
2 title: "Программирование"
3 date: 2024-02-11T15:26:15Z
4 draft: false
5 weight: 30
6 ---
7
8 ## Лабораторные работы 3 семестра
9
10 ### Лабораторная работа №1:
11
12 [1.1 Простая программа-калькулятор](...)
13
14 [1.2 Игра "Угадай число" алгоритмом бинарного поиска](...)
15
16 ### Лабораторная работа №2:
17
18 [Действия над множествами](...)
19
20 ### Лабораторная работа №3:
21 [1.1 Модернизация калькулятора (добавление точности вычислений)](...)
22
23 [1.2 Модернизация калькулятора (переменное количество неименованных аргументов)](...)
24
25 ### Лабораторная работа №4:
```

Рисунок 4. Элемент файла prog_3-4.md

После создания и наполнения всех файлов необходимо настроить сайт так, чтобы все элементы отображались корректно. Дело в том, что если запустить сайт сейчас локально, то в нем не будет видно ни деления по курсам, ни дисциплин. Завершающим шагом будет настройка файла `hugo.toml`. В разделе `[languages]` необходимо добавить подраздел `[[menu.main]]`, который будет отвечать за расположение курсов в панели меню. Создать таких нужно 5 — по одному на курс и один для расположения раздела “О себе”. Структура расположения элементов меню представлена на рисунке 5.



```

61  [[menu.main]]
62      identifier = "1 курс"
63      name      = "1 курс"
64      url       = "/first_course"
65  [[menu.main]]
66      identifier = "2 курс"
67      name      = "2 курс"
68      url       = "/second_course"
69  [[menu.main]]
70      identifier = "3 курс"
71      name      = "3 курс"
72      url       = "/third_course"
73  [[menu.main]]
74      identifier = "4 курс"
75      name      = "4 курс"
76      url       = "/forth_course"
77  [[menu.main]]
78      identifier = "0 себе"
79      name      = "0 себе"
80      url       = "/about"
81

```

Рисунок 5. Расположение основных разделов директории “content” на главной странице портфолио

Чтобы портфолио не было слишком однообразным и минималистичным стоит поменять задний фон веб-портфолио. Для этого в секцию [params] добавим элемент backgroundImage, значением которого — путь к изображению. Изображения хранятся в директории /static/images/, но путь к ним — /images/<ваше изображение>. Главная страница портфолио представлена на рисунке 6. Тема сайта по умолчанию светлая, потому что она обеспечивает лучшую читаемость и восприятие текста в условиях хорошего освещения, снижает утомляемость глаз при беглом чтении и более естественна для восприятия. Однако, имеется возможность переключиться на темную тему при помощи кнопки, расположенной после раздела “О себе”. Она реализована путем добавления в файл hugo.toml в секции [params] параметра enableThemeToggle, установленного в значении true.

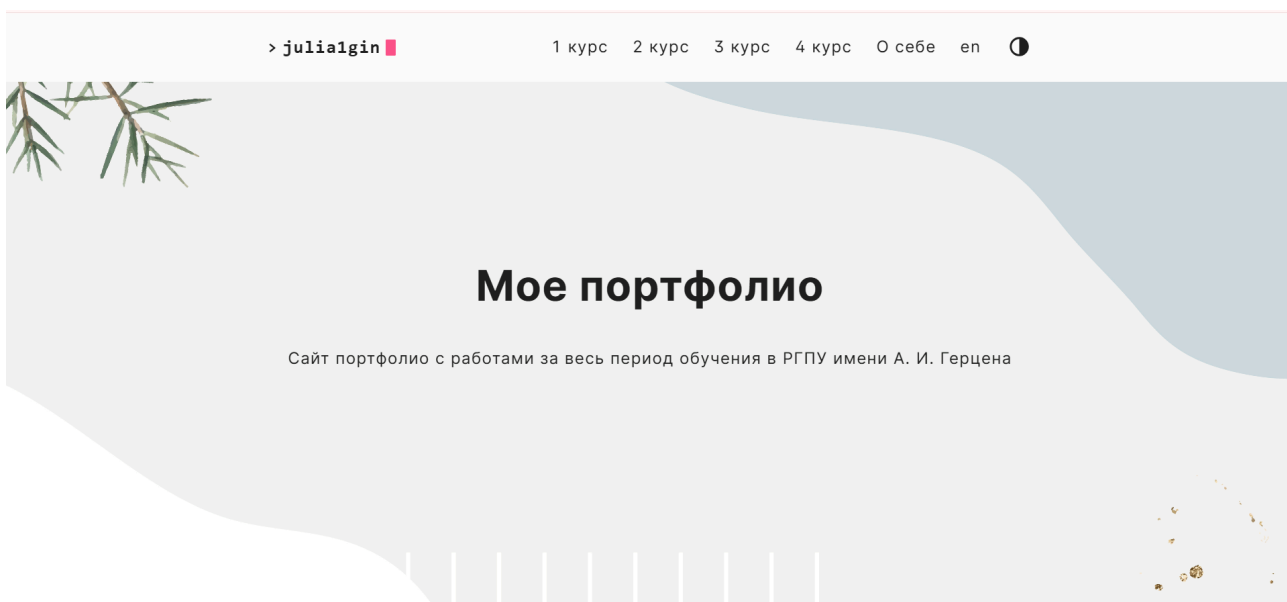


Рисунок 6. Главная страница веб-портфолио студента

Пример оформления дисциплины «Программирование» на сайте, размещенном в сети, можно увидеть на рисунке 7. Тема была переключена на темную для лучшей читаемости.

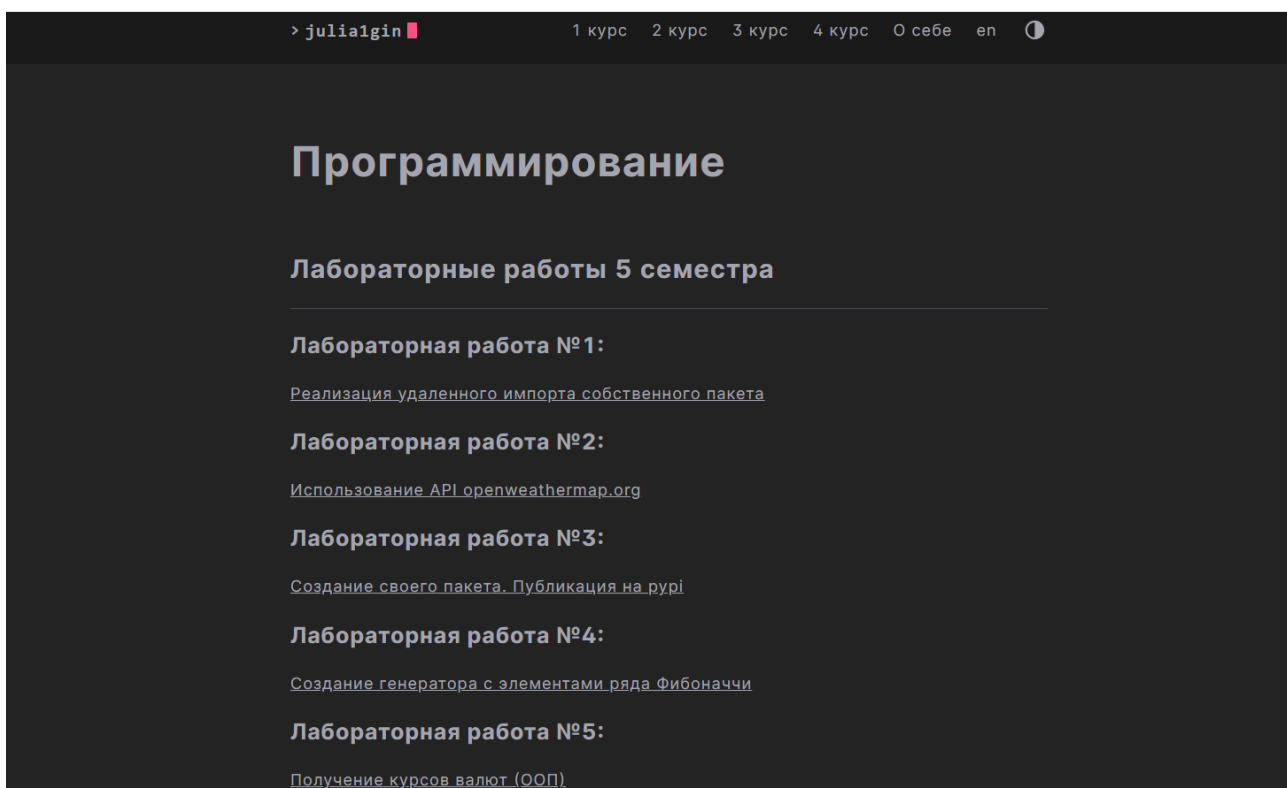


Рисунок 7. Дисциплина «Программирование» в разделе “3 курс”

2.3 Размещение сайта на Github Pages

Предварительно был создан Github репозиторий для будущего сайта. Заранее также необходимо сгенерировать и настроить ssh ключа для контроля версий, чтобы работать с git через командную строку Windows.

Чтобы опубликовать сайт портфолио на Github Pages, необходимо сохранить все изменения и выгрузить их в удаленный репозиторий. Сначала нужно выполнить команду `“git add --all .”`, чтобы добавить в индекс все изменения. Затем фиксируем все изменения командой `“git commit -m ‘Portfolio’”`. Выгрузку в удаленный репозиторий можно сделать при помощи следующих команд:

- 1) `git remote add origin git@github.com:ваш-username/ваш-repo.git`
- 2) `git branch -M main`
- 3) `git push -u origin main`

После выполнения команд можно перейти в свой удаленный Github репозиторий, обновить страницу и убедиться, что все сохраненные изменения появились. Далее необходимо выбрать раздел `“Settings”`, в котором перейти в подраздел `“Pages”`. Вместо `“Deploy from branch”` необходимо выбрать `“Github Actions”`. Из предложенных workflows следует выбрать `“Github Pages Jekyll”`. Файл переименовываем в `hugo.yaml`, затем копируем текст, представленный в приложении А, меняем содержимое файла на скопированный текст и нажимаем на кнопку `“Commit changes”`. После этого уходим в раздел `“Actions”` и наблюдаем, как деплоится наш коммит на сайт [16]. В случае успеха будет видна картина, представленная на рисунке 2.

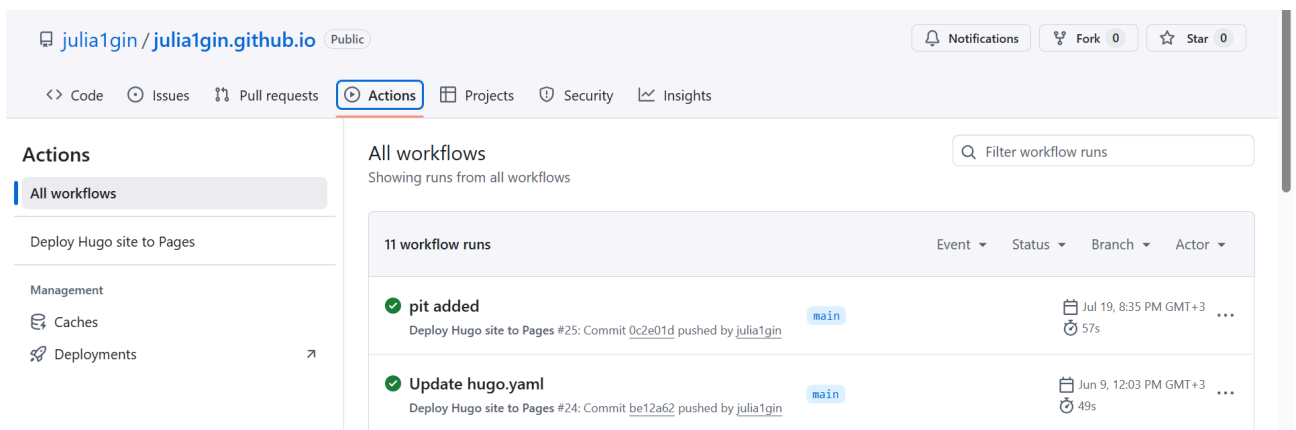


Рисунок 2. Успешное размещение сайта на Github Pages

Если кликнуть на выполненный коммит, то можно провалиться в его структуру, где мы обнаружим ссылку на размещенный в сети Интернет сайт портфолио. Структуру выполненного коммита можно увидеть на рисунке 3.

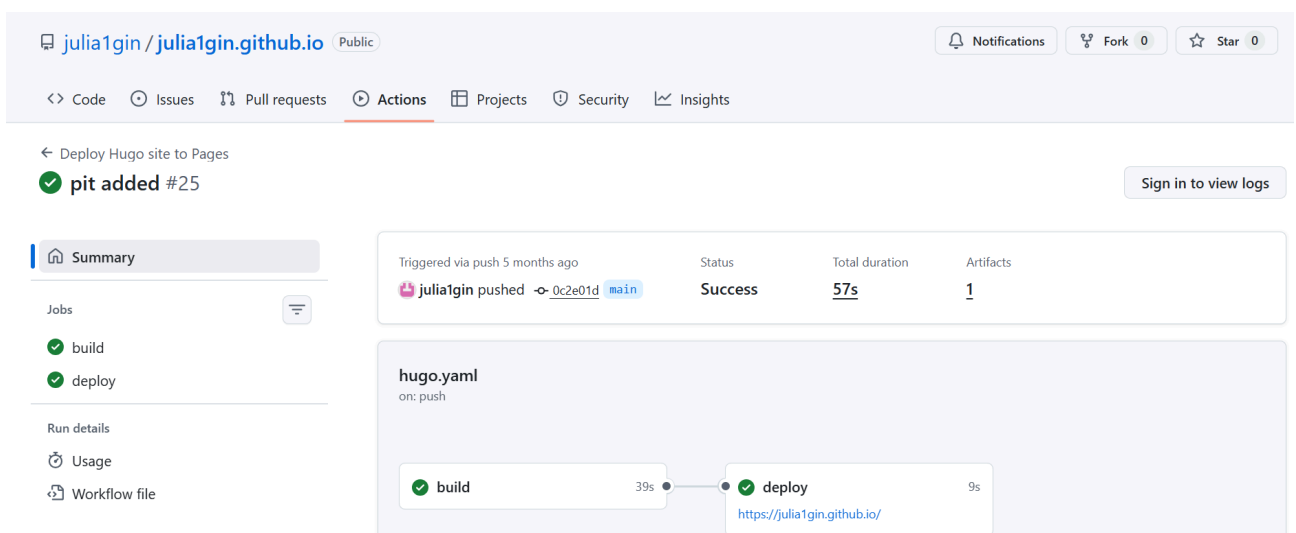


Рисунок 3. Выполненное размещение сайта портфолио в сети

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель по разработке веб-портфолио студента на примере дисциплины «Программирование» с применением генератора статических сайтов Hugo и размещению сайта в сеть при помощи Github Pages.

В теоретической части курсовой работы был проведен анализ понятия портфолио студента, рассмотрены его основные виды, функции и требования к содержанию электронного портфолио. Было установлено, что электронное портфолио позволяет не только фиксировать результаты учебной деятельности, но и демонстрировать уровень сформированности профессиональных навыков, владение языками программирования, инструментами разработки и современными информационными технологиями.

Также в рамках теоретического исследования был выполнен сравнительный анализ инструментов для создания веб-портфолио: конструкторов сайтов, систем управления контентом и генераторов статических сайтов. По результатам анализа обоснован выбор генератора статических сайтов Hugo как оптимального инструмента для студента-программиста. Данный выбор обусловлен высокой скоростью работы, бесплатной моделью распространения, простотой хостинга, гибкостью настройки и возможностью размещения сайта на доступных платформах, таких как GitHub Pages.

В практической части курсовой работы была выполнена установка и базовая настройка генератора статических сайтов Hugo, подготовлена рабочая среда и выбрана тема оформления, соответствующая требованиям к портфолио студента технического направления. Была разработана логически выстроенная структура сайта с разделением по курсам обучения и дисциплинам, что обеспечивает удобную навигацию и последовательное представление учебных материалов.

Особое внимание было уделено наполнению портфолио по дисциплине «Программирование». Лабораторные и практические работы были структурированы по семестрам и размещены с использованием специализированных платформ для хранения и демонстрации исходного кода, таких как GitHub, Replit и Google Collaboratory, что соответствует современным требованиям к представлению программных проектов.

Кроме того, была выполнена кастомизация конфигурационного файла Hugo, настройка меню навигации и визуальных параметров сайта, а также реализована возможность переключения темы оформления. Завершающим этапом работы стало размещение готового веб-портфолио в сети Интернет с использованием сервиса GitHub Pages и автоматизированного процесса деплоя.

Результатом работы является функционирующее веб-портфолио студента студента, размещенное по адресу <https://julia1gin.github.io/>, которое может быть использовано как в образовательных целях, так и в качестве профессионального инструмента для демонстрации компетенций при дальнейшем трудоустройстве. Полученные в ходе выполнения курсовой работы навыки имеют практическую значимость и могут быть применены в дальнейшей учебной и профессиональной деятельности в сфере информационных технологий.

ЛИТЕРАТУРА

1. Российские компании выбирают фрилансеров по портфолио и уровню ответственности // Cossa : [сайт]. — URL: <https://www.cossa.ru/news/343325/> (дата обращения: 24.11.2025).
2. Гостин, А. М. Создание и ведение карьерного веб-портфолио студента / А. М. Гостин, С. В. Панюкова // Высшее образование в России. — 2014. — № 4. — С. 5. — URL: <https://cyberleninka.ru/article/n/sozдание-i-vedenie-kariernogo-veb-portfolio-studenta> (дата обращения: 24.11.2025).
3. Что такое портфолио студента. Как составить. Образец // Анти-Антиплагиат : [сайт]. — URL: <https://xn----7sbbaar5acc1ard1a0beh.xn--p1ai/blog/portfolio-studenta> (дата обращения: 24.11.2025).
4. Байкина, Е. А. Роль цифрового портфолио компетенций в общей системе оценки качества результатов обучения студентов вуза / Е. А. Байкина // Высшее образование сегодня. — 2022. — № 5-6. — С. 23–29. — ISSN 1726-667X. — URL: <https://e.lanbook.com/journal/issue/326807> (дата обращения: 26.11.2025). — Режим доступа: для авториз. пользователей.
5. Галимуллина, Э. З. Методические рекомендации по созданию е-портфолио. Учебно-методическое пособие / Э. З. Галимуллина, Л. Ю. Жестков. — Елабуга : ЕИ К(П)ФУ, 2015. — 44 с. — URL: https://kpfu.ru/Metodicheskie.rekomendacii.po.sozdaniju.e_portfolio.pdf (дата обращения: 27.11.2025). — Текст : электронный.
6. Основы информационной культуры : учебное пособие / А. А. Зубрий [и др.]. — Санкт-Петербург : ИЭО СПбУТУиЭ, 2012. — ISBN 978-5-94047-492-0. — URL: <https://e.lanbook.com/book/64044> (дата обращения: 27.11.2025). — Режим доступа: для авториз. пользователей. — Текст : электронный.

7. Инструкция по созданию и заполнению электронного портфолио для обучающихся // Образовательный портал заочного, дополнительного образования ФГБОУ ВО «Саратовская государственная юридическая академия» : [сайт]. — URL: <https://portal.ssla.ru/instruction> (дата обращения: 27.11.2025).
8. Информационные технологии в социальной сфере : материалы конференции / под ред. В. Н. Аниськина [и др.]. — Самара : СГСПУ, 2017. — 154 с. — URL: <https://e.lanbook.com/book/137477> (дата обращения: 30.11.2025). — Режим доступа: для авториз. пользователей. — Текст : электронный.
9. Богатенков, С. А. Разработка сайтов и web-приложений : учебное пособие для вузов / С. А. Богатенков, Е. В. Бунова, В. В. Костерин. — Санкт-Петербург : Лань, 2025. — 96 с. — ISBN 978-5-507-52835-6. — URL: <https://e.lanbook.com/book/502463> (дата обращения: 30.11.2025). — Режим доступа: для авториз. пользователей. — Текст : электронный.
10. Diaz, C. Using Static Site Generators for Scholarly Publications and Open Educational Resources / C. Diaz // Code4Lib. — 2018. — № 42.
11. Vepsäläinen, J. Bridging Static Site Generation with the Dynamic Web / J. Vepsäläinen, P. Vuorimaa // Web Engineering. ICWE 2022. Lecture Notes in Computer Science. — Cham : Springer, 2022. — С. 437–442.
12. Создаём статичный сайт на Hugo // Код : [электронный журнал]. — URL: https://thecode.media/hugo/?utm_source=telegram&utm_medium=social&utm_campaign=post (дата обращения: 05.12.2025).
13. Официальная документация Hugo // Hugo : [сайт]. — URL: <https://gohugo.io/documentation/> (дата обращения: 06.12.2025).
14. Генерация статических сайтов с Hugo // Habr : [сайт]. — URL: <https://habr.com/ru/articles/700640/> (дата обращения: 15.12.2025).
15. Язык разметки Markdown: шпаргалка по синтаксису с примерами // Skillbox : [сайт]. — URL:

<https://skillbox.ru/media/code/yazyk-razmetki-markdown-shpargalka-po-sintaksisu-s-primerami/> (дата обращения: 17.12.2025).

16. Как установить CMS Hugo и разместить свой сайт на GitHub Pages // UnixHost : [сайт]. — URL: <https://unixhost.pro/blog/ru/2023/02/kak-ustanovit-cms-hugo-i-razmestit-svoj-sajt-na-github-pages/> (дата обращения: 20.12.2025).

ПРИЛОЖЕНИЕ А

```
name: Deploy Hugo site to Pages
on:
  push:
    branches: ["master"]
    workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: "pages"
  cancel-in-progress: false

defaults:
  run:
    shell: bash

jobs:
  build:
    runs-on: ubuntu-latest
    env:
      HUGO_VERSION: 0.147.0
    steps:
      - name: Install Hugo CLI
        run: |
          wget -O ${{ runner.temp }}/hugo.deb
          https://github.com/gohugoio/hugo/releases/download/v${HUGO_VERSION}/hugo_extended_${HUGO_
          VERSION}_linux-amd64.deb \
            && sudo dpkg -i ${{ runner.temp }}/hugo.deb
      - name: Install Dart Sass
        run: sudo snap install dart-sass
      - name: Checkout
        uses: actions/checkout@v4
        with:
          submodules: recursive
      - name: Setup Pages
        id: pages
        uses: actions/configure-pages@v5
      - name: Install Node.js dependencies
        run: "[[ -f package-lock.json || -f npm-shrinkwrap.json ]] && npm ci || true"
      - name: Build with Hugo
        env:
          HUGO_CACHEDIR: ${{ runner.temp }}/hugo_cache
          HUGO_ENVIRONMENT: production
        run: |
          hugo \
            --minify \
            --baseURL "${{ steps.pages.outputs.base_url }}"
      - name: Upload artifact
        uses: actions/upload-pages-artifact@v3
        with:
          path: ./public

  deploy:
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    runs-on: ubuntu-latest
    needs: build
    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```